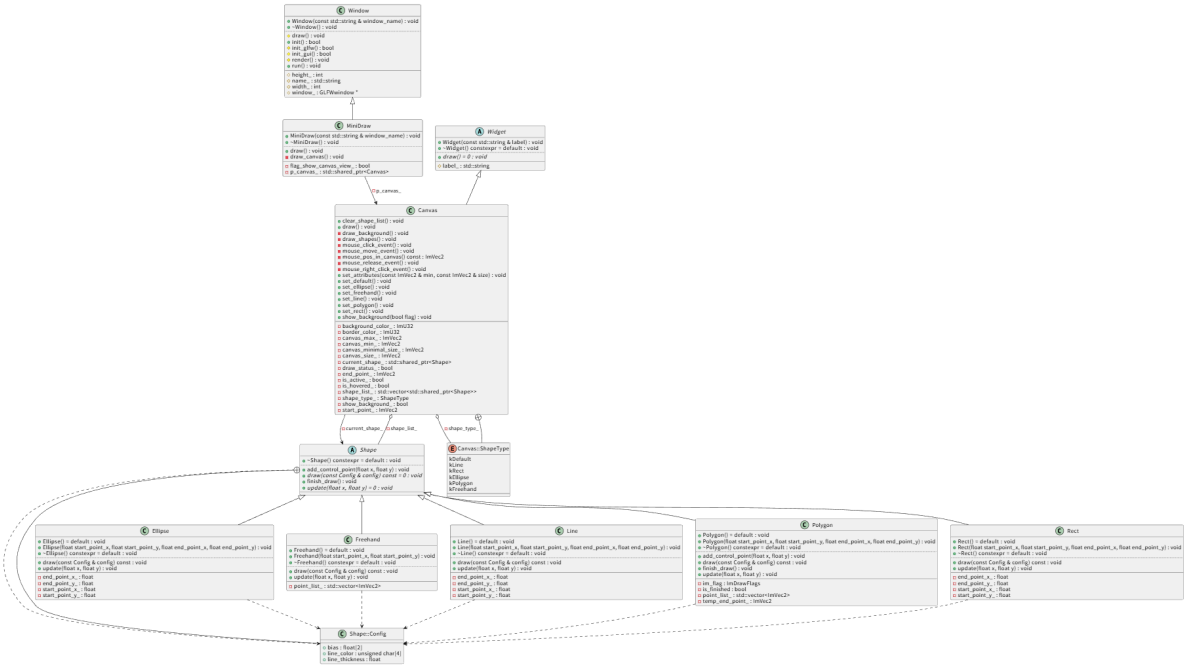
HW1-MiniDraw 作业报告

王浛丞 PB24111687

1. 项目概述

该项目的类图如下：



1.1 项目说明

我认为可以把本次 minidraw 项目分为以下三部分，从两个方向来理解

用户信息向下传递

- 用户交互部分（minidraw_window 单元文件）
  - void draw()：imgui的接口函数，调用 draw_canvas() 进行界面绘制
  - draw_canvas()： 绘制顶部 button，text 等组件

    它们负责与用户交互，并把交互信息通过 p_canvas 的接口传递给下层，主要是用户选择的图形类型
- 用户信息处理 (canvas_wiget 单元文件)
  - draw()：首先调用 draw_background() 检测鼠标按键活动并绘制画布背景

    然后在用户鼠标上绘制隐形 button，并跟据 button 的行为调用不同的 mouse_event 函数，而后跟据当前图形以及 event 类型调用 shape 的不同方法

    最后调用 draw_shapes() 在画布上绘制图案
  - 交互主要逻辑如下：

    如果用户按下左键且当前没有在绘制，则会跟据 shape_type_ 来初始化 shape

    如果用户结束绘制，则会重置状态并把当前图形 push 到 list 里存档
  - draw_background()
- 信息储存 (shape 系列文件)

    - 跟据不同情况处理，如：
        - 绘制多边形：当按下右键时，调用 finish_draw，结束绘制并填充颜色
        - 绘制多边形：当按下左键时，调用 add_control_point，添加一个控制点到 vector 中
        - 绘制椭圆：当没有按下左键、仅移动时，更新结束点的坐标

图形绘制

- minidraw_window：
    - draw => draw_canvas => p_canvas->draw
- canvas_wiget
    - draw => draw_shapes 遍历已有的 shape 列表并调用 shape 的 draw
- shape
    - draw 调用 ImDrawList 获取当前窗口信息，然后跟据 shape 类型调用不同的 add 方法进行绘制

---

2. 作业功能说明

2.1 图形绘制（ellipse、freehand）

下表总结了各类图形的绘制方式和核心逻辑：

图形类型	初始化	交互方式	更新逻辑	完成条件
ellipse	按下鼠标左键	移动鼠标绘制	`update()` 持续更新 end_point	再次点击鼠标左键
Polygon	按下鼠标左键	每次左键点击添加顶点	`add_control_point()` 添加新顶点	按下鼠标右键
Freehand	按下鼠标左键	拖动鼠标绘制	`update()` 持续添加路径点	松开鼠标左键

2.1.1 Polygon

在实现 Polygon 时，采用了 `finish_draw()` 函数来处理右键闭合，而不是传递特殊值。这样的设计更加优雅和符合面向对象的原则。

关键点：

- 使用 `point_list_` 存储已确定的顶点
- `temp_end_point_` 实现实时预览
- `is_finished` 标志控制是否完成绘制
- `im_flag` 控制是否闭合图形

详细实现见第3部分。

2.1.2 Freehand

在 Freehand 的实现中，使用 `AddPolyline` 绘制不闭合的折线，同时使用距离阈值优化：

优化策略：只有当新点与上一个点的距离超过阈值（0.1）时才添加新点，这样可以：

- 避免鼠标移动产生过多冗余点
- 减少内存占用和渲染负担
- 保持曲线平滑度

详细实现见第3部分。

2.1.3 实现要点

- **类型选择**：直接使用 ImGui 的 `ImVec2` 类型存储坐标，避免频繁类型转换
- **预编译优化**：使用 `#ifdef IMGUI_DEFINE_MATH_OPERATORS` 利用运算符重载简化代码（详见3.7节）
- **状态管理**：使用标志位（如 `is_finished`、`im_flag`）控制图形的绘制状态

2.2 设置线条颜色及粗细 & 设置填充

UI交互层（minidraw_window）：

- 使用 `ImGui::ColorEdit4` 提供颜色选择器（线条颜色和填充颜色）
- 使用 `ImGui::SliderFloat` 提供粗细调节滑块（范围 1.0-10.0）

数据管理层（canvas_widget）：

- 在 Canvas 中维护当前选择的颜色和粗细状态
- 颜色格式：`ImVec4`（RGBA，范围 0.0-1.0）

数据传递：

- 创建 Shape 时将颜色转换为 `unsigned char[4]`（0-255）
- Shape 内部转换为 `ImU32` 格式存储（打包为32位整数）

详细的颜色数据流和转换机制见第3.5节。

2.3 颜色部分实现逻辑

首先，如果要实现一个 shape 能有多个颜色（比如多边形每条边颜色不一样），这是比较麻烦的；其次，如果实现绘制好后再染色，也是比较麻烦的，因为需要判断修改的是哪个图形。

综上所述，采用比较简单的：绘制前设置，绘制时保持不变的策略。

尽管用户可以在绘制中与颜色选择组件交互，但是并不会有任何影响。

---

3. 实现细节

3.1 类设计架构

3.1.1 Shape 基类设计

Shape 类是所有图形的抽象基类，定义了图形绘制的统一接口：

```
class Shape {
protected:
    ImU32 line_color_;      // 线条颜色 (RGBA打包为32位整数)
    float line_thickness_;   // 线条粗细
    ImU32 fill_color_;       // 填充颜色
public:
    virtual void draw(const Config& config) const = 0;  // 纯虚函数，绘制图形
    virtual void update(float x, float y) = 0;          // 更新图形状态
    virtual void add_control_point(float x, float y) {} // 添加控制点（多边形用）
    virtual void finish_draw() {}                       // 完成绘制（多边形用）

    // 颜色转换辅助函数：从 unsigned char[4] 转换为 ImU32
    void set_color(ImU32 &color_, const unsigned char color[4]);
};
```

Config 结构体：封装绘制时的配置参数，主要包括坐标偏移（bias），线条颜色及粗细，填充颜色。

3.1.2 派生类详细设计

类名	数据成员	特点
Line	`start_point_x/y`, `end_point_x/y`	最简单的图形，两点确定
Rect	`start_point_x/y`, `end_point_x/y`	对角两点确定矩形
Ellipse	`start_point_x/y`, `end_point_x/y`	外接矩形的对角坐标确定椭圆
Polygon	`std::vector<ImVec2> point_list_`, `temp_end_point_`, `is_finished`, `im_flag`	顶点列表 + 状态标志
Freehand	`std::vector<ImVec2> point_list_`	连续点列表形成自由曲线

3.2 Canvas 组件核心实现

3.2.1 状态管理

Canvas 维护以下关键状态：

```
// 绘制状态
bool draw_status_ = false;              // 是否正在绘制中
ShapeType shape_type_;                  // 当前选中的图形类型
std::shared_ptr<Shape> current_shape_;  // 当前正在绘制的图形
```

```
std::vector<std::shared_ptr<Shape>> shape_list_;  // 已完成的图形列表

// 鼠标交互
ImVec2 start_point_, end_point_;  // 起点和当前点
bool is_hovered_, is_active_;     // 鼠标悬停和激活状态

// 颜色和粗细
ImVec4 current_line_color_;       // 当前线条颜色 (RGBA范围0-1)
float current_line_thickness_;    // 当前线条粗细
ImVec4 current_fill_color_;       // 当前填充颜色
```

3.2.2 draw() 主循环

Canvas 的 `draw()` 方法是整个交互逻辑的核心：

```
void Canvas::draw() {
    draw_background();  // 1. 绘制背景并创建不可见按钮

    // 2. 处理鼠标事件
    if (is_hovered_ && ImGui::IsMouseClicked(ImGuiMouseButton_Left))
        mouse_click_event();       // 左键点击
    if (is_hovered_ && ImGui::IsMouseClicked(ImGuiMouseButton_Right))
        mouse_right_click_event(); // 右键点击
    mouse_move_event();            // 鼠标移动
    if (!ImGui::IsMouseDown(ImGuiMouseButton_Left))
        mouse_release_event();     // 左键释放

    draw_shapes();  // 3. 绘制所有图形
}
```

3.2.3 不可见按钮机制

使用 ImGui 的 `InvisibleButton` 捕获画布区域的鼠标事件：

```
void Canvas::draw_background() {
    // 创建覆盖整个画布的不可见按钮
    ImGui::SetCursorScreenPos(canvas_min_);
    ImGui::InvisibleButton(
        label_.c_str(),
        canvas_size_,
        ImGuiButtonFlags_MouseButtonLeft |
ImGuiButtonFlags_MouseButtonRight);

    // 记录鼠标状态
    is_hovered_ = ImGui::IsItemHovered();  // 鼠标是否悬停在按钮上
    is_active_ = ImGui::IsItemActive();    // 按钮是否被激活
}
```

3.3 鼠标事件处理详解

3.3.1 mouse_click_event() - 左键点击

状态转换逻辑:

```
void Canvas::mouse_click_event() {
    if (!draw_status_) {  // 如果未在绘制中
        // 开始绘制：初始化新图形
        draw_status_ = true;
        start_point_ = end_point_ = mouse_pos_in_canvas();

        // 颜色转换：ImVec4 (0-1) -> unsigned char (0-255)
        unsigned char line_color[4] = {
            (unsigned char)(current_line_color_.x * 255),
            (unsigned char)(current_line_color_.y * 255),
            (unsigned char)(current_line_color_.z * 255),
            (unsigned char)(current_line_color_.w * 255)
        };
        // 填充颜色转换同理...

        // 根据图形类型创建对应实例
        switch (shape_type_) {
            case kEllipse:
                current_shape_ = std::make_shared<Ellipse>(...);
                break;
            case kPolygon:
                current_shape_ = std::make_shared<Polygon>(...);
                break;
            // ... 其他类型
        }
    }
    else {  // 如果正在绘制中
        if (shape_type_ == kPolygon) {
            // 多边形：左键添加顶点
            current_shape_->add_control_point(end_point_.x, end_point_.y);
        }
        else {
            // 其他图形：第二次点击结束绘制
            draw_status_ = false;
            shape_list_.push_back(current_shape_);
            current_shape_.reset();
        }
    }
}
```

3.3.2 mouse_move_event() - 鼠标移动

持续更新当前正在绘制的图形:

```
void Canvas::mouse_move_event() {
    if (draw_status_) {
        end_point_ = mouse_pos_in_canvas();
        if (current_shape_) {
            current_shape_->update(end_point_.x, end_point_.y);
        }
    }
}
```

3.3.3 mouse_right_click_event() - 右键点击

专门用于多边形的闭合：

```
void Canvas::mouse_right_click_event() {
    if (draw_status_ && current_shape_) {
        if (shape_type_ == kPolygon) {
            current_shape_->finish_draw();  // 设置闭合标志
            draw_status_ = false;
            shape_list_.push_back(current_shape_);
            current_shape_.reset();
        }
    }
}
```

3.3.4 mouse_release_event() - 左键释放

处理 Freehand 的绘制结束：

```
void Canvas::mouse_release_event() {
    if (draw_status_ && shape_type_ == kFreehand) {
        draw_status_ = false;
        shape_list_.push_back(current_shape_);
        current_shape_.reset();
    }
}
```

3.4 各图形类的详细实现

3.4.1 Ellipse 椭圆

核心思路：用外接矩形确定椭圆的中心和半径

```
void Ellipse::draw(const Config& config) const {
    ImDrawList* draw_list = ImGui::GetWindowDrawList();

    // 计算中心点
    ImVec2 center = ImVec2(
        config.bias[0] + (start_point_x_ + end_point_x_) / 2.0,
        config.bias[1] + (start_point_y_ + end_point_y_) / 2.0
    );

    // 计算半径（可以不相等，形成椭圆）
    ImVec2 radius = ImVec2(
```

```
        fabs(end_point_x_ - start_point_x_) / 2.0,
        fabs(end_point_y_ - start_point_y_) / 2.0
    );

    // 先填充后描边
    draw_list->AddEllipseFilled(center, radius, fill_color_, 0.0f, 0);
    draw_list->AddEllipse(center, radius, line_color_, 0.0f, 0, line_thickness_);
}

void Ellipse::update(float x, float y) {
    end_point_x_ = x;
    end_point_y_ = y;
}
```

3.4.2 Polygon 多边形

关键机制:

1. 使用 `point_list_` 存储已确定的顶点
2. `temp_end_point_` 存储鼠标当前位置，用于预览
3. `is_finished` 标志判断是否完成绘制
4. `im_flag` 控制是否闭合

```
void Polygon::draw(const Config& config) const {
    ImDrawList* draw_list = ImGui::GetWindowDrawList();
    ImVec2 bias = ImVec2(config.bias[0], config.bias[1]);

    // 坐标转换：画布坐标 -> 屏幕坐标
    std::vector<ImVec2> bias_point_list_;
    for (const auto& pt : point_list_) {
        bias_point_list_.push_back(ImVec2(bias.x + pt.x, bias.y + pt.y));
    }

    // 如果未完成，添加临时预览点
    if (!is_finished) {
        bias_point_list_.push_back(ImVec2(
            bias.x + temp_end_point_.x,
            bias.y + temp_end_point_.y
        ));
    }

    // 填充（仅在完成且点数>=3时）
    if (is_finished) {
        draw_list->AddConvexPolyFilled(
            bias_point_list_.data(),
            bias_point_list_.size(),
            fill_color_
        );
    }

    // 绘制边框（im_flag控制是否闭合）
    draw_list->AddPolyline(
        bias_point_list_.data(),
```

```
        bias_point_list_.size(),
        line_color_,
        im_flag,  // ImDrawFlags_Closed 或 ImDrawFlags_None
        line_thickness_
    );
}

void Polygon::update(float x, float y) {
    if (im_flag == ImDrawFlags_None) {
        temp_end_point_ = ImVec2(x, y);  // 更新预览点
    }
}

void Polygon::finish_draw() {
    if (point_list_.size() > 2) {
        im_flag = ImDrawFlags_Closed;  // 设置闭合标志
    }
    is_finished = true;
}

void Polygon::add_control_point(float x, float y) {
    point_list_.push_back(ImVec2(x, y));  // 添加确定的顶点
}
```

多边形绘制流程图:

```
开始 -> 左键点击(创建) -> 移动鼠标(预览) -> 左键点击(添加顶点)
     -> ... 重复 ... -> 右键点击(finish_draw) -> 完成
```

3.4.3 Freehand 自由绘制

优化点: 使用距离阈值避免点数过多

```
void Freehand::draw(const Config& config) const {
    ImDrawList* draw_list = ImGui::GetWindowDrawList();
    ImVec2 bias = ImVec2(config.bias[0], config.bias[1]);

    // 坐标转换
    std::vector<ImVec2> bias_point_list_;
    for (const auto& pt : point_list_) {
        bias_point_list_.push_back(ImVec2(bias.x + pt.x, bias.y + pt.y));
    }

    // 绘制折线（不闭合）
    draw_list->AddPolyline(
        bias_point_list_.data(),
        bias_point_list_.size(),
        line_color_,
        ImDrawFlags_None,  // 不闭合
        line_thickness_
    );
}

void Freehand::update(float x, float y) {
    ImVec2 p_curr = ImVec2(x, y);
```

```
    ImVec2 diff = ImVec2(
        p_curr.x - point_list_.back().x,
        p_curr.y - point_list_.back().y
    );

    // 只有当距离足够大时才添加新点（阈值 0.01）
    if (diff.x * diff.x + diff.y * diff.y > 0.01) {
        point_list_.push_back(p_curr);
    }
}
```

距离阈值的作用：

- 避免鼠标移动产生过多冗余点
- 减少内存占用和渲染负担
- 保持曲线平滑度

3.5 颜色管理实现

3.5.1 颜色数据流

```
UI组件 (ImVec4, 0.0-1.0)
    ↓
Canvas (存储当前颜色)
    ↓
Shape构造 (转换为 unsigned char[], 0-255)
    ↓
Shape存储 (转换为 ImU32, 打包存储)
    ↓
绘制函数 (直接使用 ImU32)
```

3.5.2 颜色转换函数

```
void Shape::set_color(ImU32 &color_, const unsigned char color[4]) {
    color_ = IM_COL32(color[0], color[1], color[2], color[3]);
}
```

IM_COL32 宏：将 4 个字节打包为 32 位整数 (R | G<<8 | B<<16 | A<<24)

3.5.3 填充控制机制

使用 **alpha 通道** 实现填充开关：

- `alpha = 0.0`：完全透明，等同于不填充
- `alpha > 0.0`：有填充效果

优势：

- 无需额外的 bool 标志
- 自然支持半透明效果
- 代码更简洁

3.6 坐标系统

3.6.1 三种坐标系

1. **屏幕坐标**：ImGui 窗口的绝对坐标
2. **画布坐标**：相对于画布左上角的坐标
3. **存储坐标**：Shape 内部存储的坐标（即画布坐标）

因此需要在绘制时做一个简单的坐标变换

3.7 ImGui 数学运算符重载

3.7.1 IMGUI_DEFINE_MATH_OPERATORS 宏

ImGui 提供了 `IMGUI_DEFINE_MATH_OPERATORS` 宏来启用 `ImVec2` 类型的数学运算符重载，使得向量运算更加直观和简洁。

启用方法：在包含 imgui.h 之前定义该宏

```
#define IMGUI_DEFINE_MATH_OPERATORS
#include <imgui.h>
```

3.7.2 本项目中的应用

在 Polygon 和 Freehand 类中，使用条件编译来兼容两种情况：

```
#ifdef IMGUI_DEFINE_MATH_OPERATORS
    // 如果定义了宏，使用运算符重载
    diff_ = p_curr_ - point_list_.back();
    bias_point_list_.push_back(bias_imvec2 + pt_);
#else
    // 否则手动计算每个分量
    diff_ = ImVec2(
        p_curr_.x - point_list_.back().x,
        p_curr_.y - point_list_.back().y
    );
    bias_point_list_.push_back(
        ImVec2(bias_imvec2.x + pt_.x, bias_imvec2.y + pt_.y)
    );
#endif
```

3.7.4 优势与注意事项

优势：

- 代码更简洁易读
- 减少手动计算错误
- 向量运算更接近数学表达式

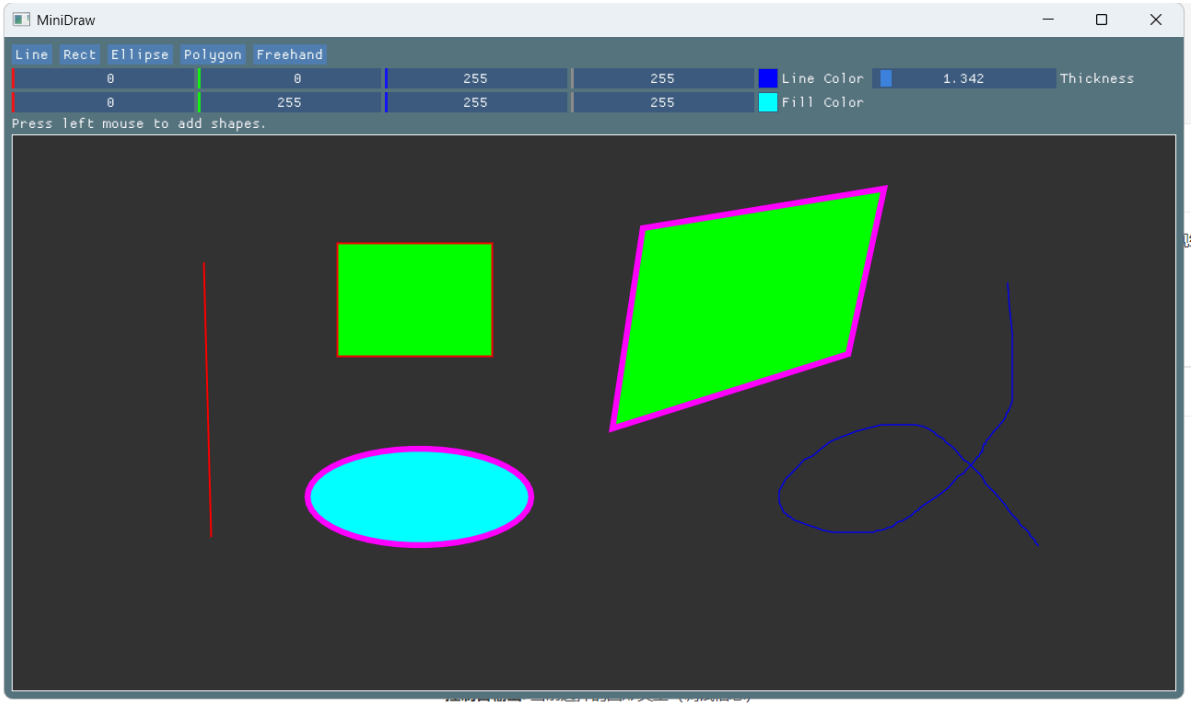
注意事项：

- 该宏并非 ImGui 的标准特性，需要显式启用
- 使用条件编译确保代码在不同环境下都能正常编译

- 支持的运算符：`+`、`-`、`*`、`/`（标量乘除）等

---

4. 测试结果



---

5. 分析与小结

4. 学习了继承和多态的使用
5. 了解了imgui的基本使用及组件